

Luxury Pulse: Fashion Intelligence Platform

Track 3: Interactive Visualization Tool

Roshni Patel (20036005)

Section-1: Executive Summary:

Luxury Pulse is a fully coded, browser-based interactive data visualization tool that consolidates global fashion and luxury retail performance data into a single, elegant four-page platform. I built it from scratch using HTML, CSS, and JavaScript without any BI tools or frameworks to demonstrate that a compelling analytical tool does not require expensive software licenses or complex infrastructure. The project targets business analysts, brand managers, and investors who need to monitor sales trends, product pricing, brand positioning, and consumer awareness across the luxury fashion industry, all in one place and accessible via any browser through a single public link.

Key Features

- **Four fully interactive pages** covering Sales Overview, Product Explorer, Brand Intelligence, and Brand Pulse each focused on a distinct business question and powered by real datasets
- **Live filtering across all page's** year, payment type, price tier, platform, and top-N brand selectors that instantly re-render every chart and KPI card without any page reload
- **16 visualizations** including a brand portfolio treemap, Google Trends seasonality heatmap, luxury price tier donut, bubble chart, multi-line trend chart, and histogram each chosen for the specific insight it surfaces rather than aesthetic preference
- **Custom Python data pipeline** that acquires, cleans, and transforms four Kaggle datasets and Google Trends API data into browser-ready JSON files, handling currency conversion, date format normalization, schema alignment across sources, and pre-aggregation for fast load times
- **Deployed on GitHub Pages** with a permanent public URL suitable for portfolio and resume use, with no login, no installation, and no dependencies required beyond a browser

Technologies Used

HTML5, CSS3, Vanilla JavaScript, Plotly.js 2.27.0, Python 3.13, pandas, pytrends, GitHub Pages

Key Findings

Through building and analysing this tool I surfaced three insights that I did not anticipate going in. First, revenue in fashion retail is strongly seasonal the monthly trend chart shows consistent concentration in specific months rather than even distribution, which has direct implications for inventory and staffing decisions. Second, the Brand Pulse heatmap revealed that the

COVID-19 pandemic did not just temporarily reduce luxury brand search interest it permanently shifted the seasonal pattern of when consumers search for these brands, a change that is still visible in 2023 and 2024 data. Third, the treemap on the Brand Intelligence page makes immediately clear that the brands with the largest product portfolios are not the most expensive volume and exclusivity sit at opposite ends of the market, and the brands that succeed at ultra-high price points deliberately maintain very small catalogues.

Impact and Value

This project demonstrates end-to-end data analyst skills in a visually distinctive domain: data acquisition via API and Kaggle, Python-based cleaning and transformation, browser-based visualization engineering, and deployment for public access. A business analyst at a luxury retailer could use this tool to benchmark category performance, identify seasonal demand patterns, and compare brand positioning without any software setup. For me personally, Luxury Pulse represents the kind of portfolio piece I can show in a job interview and walk through in real time a tool I built entirely, can explain completely, and can hand to someone with a link.

Section-2: Project Goals & Context

Problem Statement

The global luxury fashion industry generates hundreds of billions of dollars in revenue annually, yet meaningful analysis of its performance remains surprisingly difficult for most people who need it. Financial data is buried in PDF annual reports. Product pricing lives on e-commerce websites with no easy way to compare across brands. Consumer interest data exists in Google Trends but requires technical knowledge to extract and interpret. And retail transaction patterns sit in disconnected datasets that nobody has bothered to bring together in one place.

The result is that even experienced analysts working in fashion retail spend more time hunting for data than analyzing it. A brand manager at a mid-sized luxury label has no easy way to answer basic questions like: which product categories are driving revenue this year compared to last? How does our price positioning compare to competitors? Are consumers searching for our brand more or less than they were before the pandemic? These questions are answerable the data exists but the friction of finding, cleaning, and visualizing it means most people simply do not do it.

I set out to remove that friction. Luxury Pulse is my answer to the question of what happens when you bring all that data together into a single, well-designed, interactive tool that anyone can open in a browser with one click.

Project Objectives

My original goals when I submitted the project proposal were straightforward: build a multi-page executive dashboard in Tableau that tracked financial performance across major luxury conglomerates including LVMH, Kering, and Burberry. Those goals evolved significantly as I got deeper into the project.

The first shift was from Tableau to a fully coded web application. I realized early on that Tableau's drag-and-drop interface, while powerful for certain use cases, limited what I could build and more importantly limited what I could show. A coded tool that I deploy publicly on

GitHub Pages is something I fully own and can demonstrate in any context. A Tableau workbook requires the viewer to have access to Tableau Public and tells an interviewer relatively little about my technical capabilities beyond knowing how to use a BI tool.

The second shift was in data sources. My original plan to extract financial data from corporate annual reports proved far more time-consuming than anticipated the PDFs were inconsistently formatted, and the manual extraction work would have consumed most of my available time before I wrote a single line of visualization code. I replaced that approach with four publicly available Kaggle datasets and the Google Trends API via Python, which gave me immediate access to clean, structured data I could work with.

My final objectives for the project were to build a fully functional four-page interactive tool with at least sixteen working visualizations, surface at least five genuine business insights from the data, deploy the tool publicly on GitHub Pages, and produce something I could speak about confidently in a job interview. All four of those objectives were met.

Target Audience

Luxury Pulse was designed for two overlapping audiences. The primary audience is business analysts and brand managers working in the fashion and luxury retail sector people who need to monitor market trends, compare brand positioning, and understand consumer behavior without requiring a data engineering background to access the information. These users have moderate data literacy they are comfortable reading charts and interpreting trends but they are not expected to write code or run queries. The tool is designed so that a non-technical user can open it in a browser, use the dropdown filters, hover over charts for precise values, and walk away with actionable insights in under ten minutes.

The secondary audience is hiring managers and technical recruiters reviewing my portfolio. For this audience the tool serves a different purpose it demonstrates that I can execute a complete data project end to end, from raw CSV files and API calls all the way to a deployed, polished, publicly accessible product. The code is on GitHub, the tool is live at a permanent URL, and the entire pipeline from data acquisition to browser rendering is documented and reproducible.

Personal Motivation

I chose the luxury fashion industry deliberately. It is one of the few domains where data analysis, brand strategy, consumer psychology, and visual design all intersect in genuinely interesting ways. Luxury brands do not compete primarily on price or convenience they compete on perception, exclusivity, and cultural relevance, all of which show up in data in ways that are compelling to analyze. When I looked at the Brand Pulse heatmap and noticed that the seasonal pattern of consumer search interest shifted permanently after COVID, that felt like a real discovery, not just a class exercise.

Beyond the domain, I chose this project because I wanted to challenge myself technically in a direction that matters for my career. I am interested in data analyst roles that go beyond building dashboards in standard BI tools roles where being able to write clean code, build functional tools, and deploy things that work are differentiating skills. Luxury Pulse gave me the opportunity to develop all those skills simultaneously under real deadline pressure, with a real deliverable at the end. The fact that I can share a live URL of something I built from scratch,

explain every design decision, and walk through the data pipeline in an interview is exactly the kind of portfolio evidence I was trying to create when I started this course.

Section-3: Data Preparation

Data Sources

My data pipeline pulled from four distinct sources, each serving a different analytical purpose within the tool.

The first and most transactionally rich source was the Fashion Retail Sales dataset from Kaggle (uploaded by atharvasoundankar). This contained 3,400 rows of customer purchase records covering 2022 and 2023, with six columns: Customer Reference ID, Item Purchased, Purchase Amount (USD), Date Purchase, Review Rating, and Payment Method. This dataset powered the entire Sales Overview page the revenue trends, category rankings, review distribution, and payment method analysis all derive from it.

The second and third sources were the Net-a-Porter and Mr Porter product datasets from Kaggle (uploaded by justinpakzad), scraped from the two leading luxury e-commerce platforms owned by the YNAP Group. These datasets contained product listings with brand name, description, price, and product type. Net-a-Porter carries women's luxury fashion and Mr Porter carries men's, which made them natural companions. Together they contributed roughly 44,000 product records, though this was reduced significantly through cleaning and pre-aggregation before loading into the browser. These datasets powered Pages 2 and 3 the Product Explorer and Brand Intelligence pages.

The fourth source was the Luxury Products Apparel Data from Kaggle (uploaded by chitwanmanchanda), a catalog of luxury clothing items with category and subcategory labels. With around 2,000 rows it was the smallest dataset, but it provided the apparel category breakdown chart on the Product Explorer page that the Net-a-Porter data alone could not produce cleanly.

The fifth source was not a downloaded file but a live API pull. I used **pytrends**, an open-source Python library, to pull Google Trends weekly search interest data for eight major luxury brands Gucci, Louis Vuitton, Prada, Chanel, Burberry, Balenciaga, Dior, and Versace from January 2019 through December 2024. This produced approximately 576 rows covering weekly search interest scores on a relative 0–100 scale per brand. This data powered the entire Brand Pulse page.

One important limitation to note across all sources is that the Fashion Retail Sales dataset, while structured like real transaction data, is a realistic simulation rather than live commercial data. I disclosed this clearly in the tool's documentation. The Net-a-Porter and Mr Porter datasets are scraped product listings from a specific point in time rather than a live feed, meaning availability and pricing may not reflect current conditions. Google Trends scores are relative within each query a score of 100 means peak popularity for that search term, not an absolute volume figure which I documented in the dashboard's tooltip text.

Data Quality Assessment

Running initial exploratory analysis with pandas on each dataset revealed a different set of quality issues across each source.

The Fashion Retail Sales dataset's most significant problem was the date column. The dates were formatted as DD-MM-YYYY strings, which pandas was silently misreading as MM-DD-YYYY the American format it defaults to. This caused 1,646 out of 3,400 rows to parse as null dates, since values like 23-03-2023 have no valid month 23. The issue was invisible until I checked null counts explicitly. Review scores also had a small number of values outside the valid 1–5 range that needed clamping.

The Net-a-Porter and Mr Porter datasets had classic web-scraping problems. Prices were stored as strings with currency symbols embedded values like "£1,240.00" needed the pound sign and comma stripped before they could be parsed as numbers. All prices were in GBP, requiring currency conversion to USD. Brand names had inconsistent casing and abbreviations "Saint Laurent," "YSL," and "Yves Saint Laurent" all appeared as separate entities for the same brand. The datasets also contained a significant number of exact duplicate rows from the scraping process.

The Luxury Apparel dataset had an unnamed index column that pandas had added during the original export, which needed to be dropped. More surprisingly, it had no brand column at all only category and subcategory fields which meant I could not use it for brand-level analysis and had to limit its use to category breakdowns.

The pytrends Google Trends data was structurally clean but required reshaping. The API returns data in wide format with one column per brand, but Plotly works best with long format data where each row represents one brand on one date. This required a melt operation to transpose the structure.

Data Cleaning and Transformation Pipeline

I built a single Python script called `clean_pipeline.py` that processes all five sources sequentially and exports twelve pre-aggregated JSON files to a `json_data/` folder. The pipeline ran end to end in under three minutes.

Step 1: Column standardization. Every dataset went through a `snake_case` normalization function that stripped whitespace, converted to lowercase, and replaced spaces and special characters with underscores. This meant I could write consistent field references across all downstream code without worrying about capitalization or spacing differences between source files.

Step 2: Date parsing fix. For the Fashion Retail Sales dataset, I passed `dayfirst=True` to `pd.to_datetime()`, which resolved all 1,646 null dates immediately. After fixing this I extracted four derived columns `year`, `month`, `month_name`, and `quarter` as separate fields. This allowed Plotly to group by time directly without any in-browser computation. I also converted the parsed dates to YYYY-MM-DD strings for JSON export, since Plotly's time-series axis requires that specific format.

Step 3: Price cleaning and currency conversion. For both Net-a-Porter files I wrote a `clean_price()` function that stripped any non-numeric characters using a regex pattern and then cast to float. After stripping I multiplied all GBP prices by 1.244, the 2023 Federal Reserve annual average exchange rate, and stored the result as `price_usd`. I also assigned a `price_tier` label to every product based on price thresholds: Entry Luxury under \$200, Mid Luxury \$200–\$800, High Luxury \$800–\$2,500, and Ultra Luxury above \$2,500.

Step 4: Brand name standardization. I built a mapping dictionary that resolved known variants to canonical brand names "Saint Laurent" and "YSL" both mapped to "Yves Saint Laurent," "MCM" normalized from "Mcm," and so on. I applied `.str.strip().str.title()` to all brand columns first to handle casing inconsistencies before applying the explicit mapping.

Step 5: Deduplication and null removal. I called `drop_duplicates()` on all datasets after cleaning and `dropna()` on critical fields dates and revenue for the sales data, prices for the product data, and brand names for the apparel data.

Step 6: Pre-aggregation. Rather than loading raw row-level data into the browser, I pre-aggregated each dataset into summary tables before exporting to JSON. For the Net-a-Porter data this reduced the file size from 43,500 rows to roughly 2,900 grouped summary rows while retaining all the information the charts needed average price, minimum, maximum, and product count grouped by brand, category, platform, and price tier. This was the single biggest performance improvement in the project.

Step 7: Google Trends reshape. After pulling trends data from `pytrends` in batches of four brands per request with a ten-second delay between calls to avoid rate limiting, I applied `df.melt()` to convert from wide to long format and extracted year, month, month_name, and quarter as derived columns before export.

Before and After Cleaning

Fashion Retail Sales — Date column

State	Date Purchase	Year	Month	Rows with null date
Before	05-02-2023 (string, misread as May 2)	—	—	1,646
After	2023-02-05 (YYYY-MM-DD string)	2023	2	0

Net-a-Porter — Price column

State	Price	Price USD	Price Tier
Before	£1,240.00 (string with symbol)	—	—
After	1,242.56 (float, GBP × 1.244)	1,242.56	High Luxury

Tools Used

The entire pipeline was written in **Python 3.13** using **pandas** for all data manipulation, **pytrends** for Google Trends API access, and the built-in **os** and **json** modules for file management. I used **Jupyter Notebook** for initial exploratory analysis and column inspection before writing the final pipeline script. No Excel or manual editing was involved at any stage every transformation is reproducible by running `python clean_pipeline.py` from the project directory.

Section-4: Design Decisions

Visualization Choices

Every chart in Luxury Pulse was selected to answer a specific business question. I made a deliberate effort throughout the project to avoid adding visualizations simply because they looked interesting each one had to justify its presence by revealing something that a different chart type would not.

Monthly Revenue Trend - Line Chart with Area Fill

For the primary revenue view, I chose a line chart rather than a bar chart because the question it answers is fundamentally about shape over time rather than magnitude at a point in time. A bar chart would have shown the same data but would have made month-to-month comparison harder the eye naturally follows a line across a time axis and picks up acceleration, deceleration, and seasonal cycles far more efficiently than it compares bar heights. I added an area fill below the line at low opacity to give the chart visual weight and reinforce the sense of accumulation, without obscuring the shape of the line itself. I considered a dual-axis chart that overlaid 2022 and 2023 data for direct year-over-year comparison but decided against it because it added complexity without adding clarity the year filter already serves that purpose by letting the user isolate each year independently.

Payment Method Distribution - Donut Chart

I chose a donut chart over a standard pie chart for two reasons. First, the hollow center creates space for a text annotation that labels the chart's subject without adding an external title, which keeps the visual compact. Second, a donut chart visually deemphasizes the absolute angle of each segment which humans read poorly and draws attention instead to the relative arc lengths, which are easier to compare. I considered a horizontal bar chart for this data, which would have been more precise, but the payment method data has only two to three categories and the proportional story is more intuitive than a ranked comparison in this context. The text labels use black font rather than the default grey specifically because the dark card background made light-coloured labels invisible a practical accessibility fix discovered during testing.

Top Categories by Revenue - Horizontal Bar Chart

I chose a horizontal bar chart over a vertical one because the category names are multi-word strings that would have required angled or rotated labels on a vertical axis, reducing readability. Horizontal bars allow full label text to display cleanly on the Y axis. I sorted the bars descending by revenue and applied a gold colour gradient full opacity at the top fading to roughly 30% opacity at the bottom so that the visual hierarchy of the ranking is encoded in both position and colour intensity simultaneously. A viewer can identify the top category immediately without reading any numbers. I considered a treemap for this chart but decided against it because category names vary significantly in character length, which causes treemap labels to become unreadable in smaller rectangles.

Review Score Distribution - Histogram

The histogram was the natural choice for showing how review scores distribute across all 2,750 transactions. I binned scores into 0.5-point intervals and used a two-colour encoding dark neutral for scores below four stars, gold for four and above to immediately communicate the

threshold between unsatisfactory and satisfactory. This makes the skew toward high ratings instantly visible without requiring the viewer to read individual bin heights. I considered a box plot, which would have shown median and quartile information more precisely, but a box plot requires statistical literacy to interpret correctly and the primary audience for this tool is not necessarily statistically trained.

Revenue by Category Over Time - Multi-Line Chart

This chart answers a question that the bar chart above cannot: not just which categories are largest, but whether they are growing or shrinking and whether their seasonal patterns differ from each other. Five lines representing the five highest-revenue categories are plotted across months, each in a distinct colour from my palette. I deliberately limited this to five lines because beyond that the chart becomes unreadable too many overlapping lines destroy the pattern that makes the chart useful. I considered a stacked area chart, which would have shown the combined total more clearly, but stacking makes individual line shapes impossible to follow, which defeats the purpose of showing trends rather than totals.

Brand Portfolio Map - Treemap

The treemap on the Brand Intelligence page replaced a scatter plot that had become unreadable with 25 brands plotted simultaneously. The scatter plot encoded average price on the X axis, maximum price on the Y axis, and product count in bubble size three dimensions of information that worked in theory but produced so many overlapping labels in practice that the chart communicated almost nothing. The treemap solved this by encoding product count in rectangle area, which human perception reads quite accurately for relative comparisons, and average price in colour intensity using a blue-to-gold gradient. Every brand gets its own clearly bounded rectangle with a label and price annotation. The trade-off is that the treemap does not show maximum price at all, but I decided that simplifying to two clean variables was better than preserving three noisy ones.

Brand Pulse Heatmap - Heatmap

The seasonality heatmap was the most deliberate chart choice in the entire project. The same data is already shown in the line chart above it monthly search interest over time, but the line chart makes seasonal patterns very difficult to see because the time axis spans five years and the monthly cycles get compressed. The heatmap reorganizes the same data into a brand-by-month grid where each cell shows the average interest for that brand in that calendar month across all years. This structure makes seasonal cycles immediately visible the December spikes appear as a column of bright gold cells, and the unexpected June-July pattern shows up as a second column of elevated colour that would be nearly invisible in the line chart. I considered a calendar heatmap instead, which plots data day by day in a calendar grid, but that format works better for a single time series than for comparing eight brands simultaneously.

Visual Encoding

The colour palette throughout Luxury Pulse is built around two primaries a warm gold (#c9a84c) and a muted steel blue (#7eb8c9) on a near-black background (#080808). This palette was chosen deliberately to reflect the aesthetic language of the luxury fashion industry, where dark backgrounds and metallic accents are standard. Beyond aesthetics, the two-colour system serves a functional purpose: gold encodes high values, importance, and positive

indicators, while blue encodes secondary or comparative data. This consistent mapping means a user who has spent a few minutes with the tool instinctively reads gold as "noteworthy" without needing a legend on every chart.

For charts that use sequential colour scales the bubble chart, treemap, and brand scatter I used a dark blue to gold gradient. This scale works because both endpoints are visually distinct from the dark background, unlike the original dark-green-to-gold scale I tested first, where the low end blended into the card colour and made data points invisible. I verified colour contrast by checking each chart on both a calibrated monitor and a lower-quality laptop screen.

One limitation I acknowledge is that the gold-and-blue palette is not fully deuteranopia-safe. A viewer with red-green colour blindness would still read the charts correctly since gold and blue are distinguishable under most forms of colour vision deficiency. However, I did not formally test the palette with a colour blindness simulator that is a planned improvement for future iterations.

Typography uses two typefaces loaded from Google Fonts. Cormorant Garamond at light weight handles all display text page titles, KPI values, and the brand name in the header because its editorial quality evokes the visual language of luxury fashion magazines. DM Sans handles all functional text labels, filters, tooltips, and chart annotations because it is highly legible at small sizes on dark backgrounds. The combination of a serif display face and a sans-serif functional face is a deliberate typographic hierarchy that signals which text demands attention and which text provides context.

Typography uses two typefaces loaded from Google Fonts. Cormorant Garamond at light weight handles all display text page titles, KPI values, and the brand name in the header because its editorial quality evokes the visual language of luxury fashion magazines. DM Sans handles all functional text labels, filters, tooltips, and chart annotations because it is highly legible at small sizes on dark backgrounds. The combination of a serif display face and a sans-serif functional face is a deliberate typographic hierarchy that signals which text demands attention and which text provides context.

Interactivity

Every filter in Luxury Pulse is linked to a JavaScript function that re-reads the data, recalculates all aggregations, and re-renders every chart on the page simultaneously. This means filters are truly interactive rather than just cosmetic changing the year filter does not just update one chart, it updates the KPI cards, the insight strip, and all five charts at the same time with no page reload.

I chose dropdown filters over sliders or checkboxes for two reasons. First, the available filter values are discrete categories specific years, specific payment methods, specific price tiers rather than continuous ranges, making dropdowns semantically appropriate. Second, dropdowns are the most universally understood UI control and require no learning curve for a new user.

Hover tooltips are enabled on every data point across all charts using Plotly's built-in hover system. Tooltips show precise numerical values formatted consistently dollar amounts with thousand separators and two-decimal precision, review scores to one decimal place, and search interest scores to one decimal place. The Brand Pulse line chart uses unified hover mode, which

shows all eight brand values simultaneously when hovering at any point on the time axis, making cross-brand comparison at a specific date effortless.

The most considered interactivity decision was the tooltip fix on the Brand Pulse line chart. When hovering near the top of the chart with eight brands in the tooltip, the tooltip box would clip at the chart boundary and cut off the first brand in the list. I resolved this by adding 120 pixels of top margin to the chart layout and configuring the hoverlabel with explicit dark background and gold border styling. This fix was small in code terms — two layout properties but it was the difference between a chart that communicated correctly and one that appeared broken at precisely the moment a user was trying to read detailed information.

Section-5: Technical Implementation

Technology Stack

Luxury Pulse is built entirely with HTML5, CSS3, and Vanilla JavaScript on the front end, with Python 3.13 and pandas handling all data preparation on the back end. I made a deliberate choice to use no JavaScript frameworks no React, no Vue, no Angular and no CSS preprocessors. Every feature was built from first principles.

I chose this approach for three reasons. First, it forces a deeper understanding of how the web works when something breaks there is no framework abstraction layer to hide behind. Second, a zero-dependency front end means the project has no npm, no build step, no node_modules folder, and no version compatibility issue. Anyone can clone the repository and open index.html with Live Server and it works immediately. Third, from a portfolio standpoint, a hiring manager looking at the code sees clean, readable JavaScript that demonstrates I understand the language rather than a configuration of someone else's abstractions.

For charting I used Plotly.js 2.27.0, loaded from the Cloudflare CDN with a single script tag. I considered D3.js, which offers more customization, but D3 requires writing significantly more boilerplate code for standard chart types. Given the scope of the project sixteen charts across four pages Plotly's declarative API was the more practical choice. It handles all the SVG rendering, zoom, pan, and hover interactions natively. I also considered Chart.js but ruled it out because it does not support treemaps or heatmaps natively, which were two of my most important chart types.

For deployment I used **GitHub Pages**, which serves the static files directly from the repository. This required no server configuration, no environment variables, and no ongoing maintenance. The only constraint is that GitHub Pages only supports static files, which is exactly what my project produces a self-contained folder of HTML, CSS, JavaScript, and JSON.

Architecture

The application has a simple three-layer architecture. The data layer consists of twelve pre-processed JSON files in the json_data/ folder. The logic layer is a single app.js file containing all data loading, filtering, aggregation, and chart rendering functions. The presentation layer is index.html and style.css, which define the page structure and visual styling.

The data flow works as follows. When the page loads in the browser, the DOMContentLoaded event fires and calls the loadAll() function. This function uses JavaScript's fetch() API to request each JSON file from the server asynchronously. Sales data loads first and immediately

render Page-1, so the user sees content within two to three seconds. Product and brand data loads in the background while the user is reading the sales charts. Once all data is loaded, the global DATA object contains every dataset in memory, and all four pages can render instantly when the user navigates between them.

When a user changes a filter dropdown, an onchange event calls the relevant render function `renderSales()`, `renderProducts()`, `renderBrands()`, or `renderPulse()`. That function reads the current filter values, filters the in-memory data, accordingly, recalculates all aggregations, and calls `Plotly.newPlot()` on each chart container. The DOM is updated entirely in memory and Plotly redraws the SVG no server requests, no page reload.

Key Technical Implementations

The `grp()` helper function is the most reused piece of logic in the entire codebase. JavaScript has no native equivalent of pandas `groupby()`, so I built my own. Given an array of objects, a key to group by, and a dictionary of aggregation functions, it returns a new array with one object per unique key value containing the aggregated results. Every chart that requires grouped data monthly revenue, category totals, brand averages call this function. Without it every chart would have required twenty or thirty lines of manual array manipulation.

Pre-aggregation in Python was the most impactful performance decision in the project. The raw Net-a-Porter dataset contained 43,500 rows. Loading and filtering that many rows in JavaScript every time a user changes a dropdown would have been noticeably slow. Instead, my Python pipeline groups the data by brand, category, platform, and price tier before export, reducing the JSON file from tens of thousands of rows to roughly 2,900 summary rows. The browser receives exactly the level of detail it needs and nothing more. This single change reduced the Product Explorer page load time from several seconds to under one second.

Sequential data loading in `loadAll()` replaced my original `Promise.all()` approach after I discovered that `Promise.all()` would silently fail and leave the entire tool blank if any single JSON file failed to load. The sequential approach loads sales data first and renders Page 1 immediately, then loads the remaining files one by one in the background. The user sees content in under three seconds even on a slow connection, and a failure in one file does not block the others.

Code Snippets

Snippet 1: The `grp()` grouping helper

```
1. function grp(arr, key, aggs) {
2.   const map = {};
3.   arr.forEach(r => {
4.     const k = r[key];
5.     if (!k) return;
6.     if (!map[k]) map[k] = { [key]: k, _r: [] };
7.     map[k]._r.push(r);
8.   });
9.   return Object.values(map).map(g => {
10.    const out = { [key]: g[key] };
11.    Object.entries(aggs).forEach(([f, fn]) => {
12.      const v = g._r.map(r => parseFloat(r[f])).filter(v =>
```

```

!isNaN(v));
13.      out[f] = fn === 'sum'    ? v.reduce((a, b) => a + b, 0)
14.          : fn === 'mean'    ? v.reduce((a, b) => a + b, 0) /
(v.length || 1)
15.          : fn === 'count'   ? g._r.length
16.          : fn === 'max'     ? Math.max(...v) : 0;
17.    });
18.    return out;
19.  });
20. }

```

This function is called with three arguments: the data array, the field to group by, and an object mapping field names to aggregation types. For example, `grp(sales, 'month_name', { order_value_usd: 'sum' })` returns one object per month with the total revenue for that month. The function handles nulls, non-numeric values, and empty groups without errors. It was the single most valuable piece of reusable logic I wrote it appears in twelve different places across the codebase.

Snippet 2: Sequential data loading with immediate render

```

async function loadAll() {
  DATA.sales = await loadJSON('clean_sales.json');
  DATA.payDat = await loadJSON('clean_sales_by_payment.json');

  // Render Page 1 immediately with sales data
  renderSales();

  // Load remaining data in the background
  DATA.nap = await loadJSON('clean_netaporter.json');
  DATA.napBrands = await loadJSON('clean_netaporter_brands.json');
  DATA.trends = await loadJSON('clean_trends.json');
  DATA.trendsYr = await loadJSON('clean_trends_yearly.json');

  document.getElementById('data-status').textContent =
    `${total.toLocaleString()} records loaded`;
}

```

The `await` keyword pauses execution until each file finishes loading before moving to the next line. By calling `renderSales()` after the first two files load before waiting for the product and trends data the user sees five working charts within two to three seconds of opening the page. This is a meaningful user experience improvement over the alternative of waiting for all twelve files to load before showing anything. The `async` keyword on the function declaration is what allows `await` to work inside it this is part of JavaScript's modern asynchronous programming model.

Section-6: Key Findings & Insights

Major Findings

Finding 1: Fashion retail revenue is sharply seasonal, not evenly distributed

The monthly revenue trend chart on the Sales Overview page makes immediately clear that spending in the Fashion Retail Sales dataset is not spread evenly across the year. There are consistent peaks concentrated in a small number of months and noticeable troughs in between. When filtered to 2022 alone versus 2023 alone, the seasonal shape shifts the peak months are not identical year over year which suggests that the pattern is influenced by factors beyond simple calendar seasonality like holidays. The practical implication for a retailer is that inventory decisions, staffing, and promotional spend should be calibrated to these peaks rather than distributed uniformly across the year.

Finding 2: A small number of categories drive the majority of revenue

The top categories bar chart consistently shows that three to four product categories account for a disproportionate share of total revenue regardless of which year or payment filter is applied. The remaining eight to nine categories in the top twelve contribute significantly less per category. This is a textbook long-tail distribution, and it has direct strategic implications a brand manager looking at this chart can immediately identify where to concentrate marketing investment and where to consider rationalizing the product mix. What surprised me about this finding was how stable the ranking was across years the top categories in 2022 remain the top categories in 2023, suggesting these are structural preferences rather than trend-driven spikes.

Finding 3: COVID permanently altered luxury brand search seasonality

This was the most unexpected and analytically interesting finding in the entire project. The Brand Pulse heatmap compares average monthly search interest across brands for all years from 2019 to 2024. When I filtered the view to 2019 alone pre-pandemic and compared the seasonal pattern to 2022 and 2023, the shape of the heatmap changed noticeably. The December concentration of search interest is present in both periods, but the mid-year pattern particularly the June and July column brightening visible in post-pandemic data is largely absent in 2019. This suggests that COVID did not just temporarily suppress luxury brand interest. It restructured when consumers engage with these brands throughout the year, a shift that persisted three and four years after the initial disruption.

Finding 4: Watches command average prices tend to fifteen times higher than fashion categories

The average price by category chart on the Product Explorer page shows a dramatic price disparity between product types. Watches sit at the far right of the chart with average prices in the range of \$7,000 to \$8,000, while clothing, shoes, and accessories cluster between \$500 and \$2,000. This gap is not surprising in isolation watch enthusiasts already know this but seeing it visualized alongside fashion categories makes the scale of the disparity concrete. It also explains why watch brands dominate the upper right quadrant of the Brand Intelligence treemap despite having relatively few products: each individual product commands a price point that fashion items simply do not reach.

Finding 5: Brand portfolio size and price exclusivity are inversely related

The Brand Intelligence treemap encodes product count in rectangle area and average price in color intensity. The most immediately striking pattern in that chart is that the brands with the largest rectangles Saint Laurent, Gucci, and similar houses with broad accessible product lines tend to have cooler, lower-intensity colors, indicating lower average prices. The brands with the most intense gold coloring Vacheron Constantin, Bovet, Jacob and Co have tiny rectangles, meaning very few products at extraordinarily high average prices. This inverse relationship between portfolio breadth and price exclusivity is a well-known principle in luxury brand strategy, and seeing it emerge clearly from scraped product data without any manual curation felt like a genuine validation of the dataset's quality

Patterns and Trends

Looking across all four pages together, a theme emerges that I did not explicitly set out to find: the luxury market is bifurcating. On one side there is a volume-oriented segment brands with large, accessible product catalogues, mid-range luxury price points, and consistent consumer search interest. On the other side there is an exclusivity-oriented segment brands with tiny catalogues, extreme price points, and more volatile search patterns. The mid-tier of this market appears relatively thin, which aligns with broader industry commentary about the disappearing middle in luxury retail.

The relationship between search interest and presumed purchasing behavior is also worth noting. Gucci consistently shows the highest or near-highest Google Trends search interest across all years in the Brand Pulse charts. However, Gucci does not appear as the highest-priced brand in the Brand Intelligence charts that position belongs to ultra-high watchmakers. This gap between awareness and price positioning suggests that the most searched brands are not necessarily the most exclusive, and that consumer curiosity and consumer purchasing power operate somewhat independently in the luxury segment.

Implications

For a brand manager at a fashion retailer, the seasonal concentration finding suggests that planning cycles should be restructured around the actual shape of demand rather than a uniform twelve-month assumption. The category concentration finding argues for deliberate portfolio rationalization investing deeper in proven top categories rather than spreading resources across a long tail of underperforming ones.

For a luxury strategy analyst, the COVID seasonality shift is the most actionable finding. If the mid-year search interest pattern is genuinely structural rather than a temporary anomaly, marketing calendars and campaign timing built on pre-2020 assumptions may be systematically mistimed. Verifying this pattern against actual sales data and adjusting campaign scheduling accordingly could be a meaningful competitive advantage.

For an investor evaluating luxury brands, the inverse relationship between portfolio breadth and price exclusivity visible in the treemap provides a quick visual framework for positioning analysis though it should be treated as a starting point for investigation rather than a definitive conclusion.

Limitations

Several important limitations constrain how strongly these findings should be interpreted. The Fashion Retail Sales dataset is a realistic simulation rather than actual commercial transaction data, which means the revenue and category patterns it shows reflect a modeled version of retail behavior rather than verified real-world outcomes. I disclosed this clearly in the tool, but it means the Sales Overview findings should be understood as illustrative of the kind of patterns that exist in real fashion retail data rather than as verified observations about any specific retailer.

The Google Trends data is relative rather than absolute. A score of 100 means peak search interest for that brand within the time queried, not a specific volume of searches. This makes cross-brand comparisons directionally informative but not precisely comparable a brand scoring 60 did not necessarily receive 60% of the searches that a brand scoring 100 received.

The Net-a-Porter and Mr Porter datasets represent product listings from a specific point in time rather than a longitudinal view of product availability or pricing. Price changes, discontinued products, and newly launched items are not reflected, which means the price positioning analysis in Pages 2 and 3 should be understood as a snapshot rather than a trend.

Finally, the dataset covers only brands carried on Net-a-Porter and Mr Porter, which skews toward contemporary luxury fashion and high-end watches. Brands that distribute through different channels directly, through department stores, or through their own retail networks are underrepresented or absent entirely. Any conclusions about the luxury market should account for this selection bias.

Section-7: Challenges & Solutions

Challenge 1: Silent Date Parsing Failure

When I first ran my cleaning pipeline on the Fashion Retail Sales dataset and checked null counts, nearly half the date column 1,646 out of 3,400 rows had silently failed to parse and returned null values. The dates looked completely valid in the raw CSV, formatted consistently as strings like 23-03-2023, so I spent a significant amount of time assuming the problem was encoding, whitespace, or some hidden character I was not seeing. I tried stripping whitespace, converting the column to string before parsing, and specifying the format explicitly as %d-%m-%Y. That last attempt worked for most rows but still produced nulls on ambiguous dates like 01-02-2023 where day and month were interchangeable.

The actual problem was much simpler once I understood it. Pandas defaults to reading dates as MM-DD-YYYY the American format but my dataset used DD-MM-YYYY, the format standard in most of the world outside the United States. A date like 23-03-2023 fails silently because there is no 23rd month and pandas returns NaT instead of raising an error. The fix was a single keyword argument: `dayfirst=True` in `pd.to_datetime()`. All 2,750 valid rows parsed correctly immediately.

What I learned from this was to never trust silent failures. Pandas will happily return a column full of nulls without any warning if the data does not match its assumptions. Checking null counts explicitly after every transformation step not just at the end of the pipeline became a non-negotiable part of my workflow after this experience.

Challenge 2: Browser Loading Stall from Large JSON Files

After completing the Python pipeline and opening the tool in the browser, the status bar showed "Loading data..." and stayed there indefinitely. The charts never rendered. This was one of the most frustrating problems I encountered because the files existed, the paths were correct, and there were no obvious JavaScript errors in the console. The issue turned out to be two separate problems compounding each other.

The first was that `clean_apparel.json` was 2MB and `clean_sales.json` was 852KB sizes that are manageable for a server but caused the browser to stall when loading all twelve JSON files simultaneously using `Promise.all()`. The second problem was that my original `loadAll()` function used `Promise.all()`, which means if any single file failed to load, the entire promise rejected and nothing rendered. One missing file `clean_apparel_by_brand.json` did not exist yet was silently blocking everything.

I solved both problems separately. For the file size issue, I rewrote the Python pipeline to export pre-aggregated summary tables instead of raw row-level data, reducing the Net-a-Porter JSON from 43,500 rows to roughly 2,900 summary rows. For the loading architecture I replaced `Promise.all()` with sequential `await` calls that loaded sales data first, rendered Page 1 immediately, and then loaded the remaining files in the background. I also created the missing JSON file. After both fixes the tool loaded Page 1 in under three seconds.

What I learned was that frontend performance is not just about JavaScript it starts with data preparation. The decision of what to aggregate in Python versus what to compute in the browser has a direct and significant impact on user experience.

Challenge 3: Scope Management and the Track Switch

My original proposal was a Tableau dashboard tracking financial performance across publicly traded luxury conglomerates including LVMH, Kering, and Burberry. Within the first week of execution, it became clear that this plan was not going to work on the available timeline. Extracting revenue data from corporate annual reports in PDF format was taking far longer than anticipated the tables were inconsistently formatted across companies and years, and what I estimated as a two- or three-hour task was expanding toward ten or fifteen hours without a clear endpoint.

I faced a choice between continuing with the original plan at the cost of everything else or making a deliberate scope change while there was still time to build something complete. I chose to change both the track and the data sources switching from Track 1 and Tableau to Track 3 and a coded JavaScript tool and replacing the annual report data with the Kaggle datasets and Google Trends API that I could access immediately. This was not a comfortable decision because it meant abandoning significant planning work, but it was the right one.

What I learned was that the most dangerous scope problems are the ones that look like they are almost solved. The annual report extraction never felt completely hopeless, which made it hard to abandon. Recognising when a task is expanding beyond its original estimate and making a clean decision to change direction rather than spending another hour hoping it resolves is a project management skill that I will apply to every future project.

Challenge 4: Chart Readability on a Dark Background

The luxury aesthetic I chose for the tool near-black backgrounds, dark card surfaces, gold accents looked elegant in the CSS but created serious readability problems when Plotly charts were placed on top of it. Plotly's default color scheme assumes a white or light background. Axis tick labels rendered in black and were completely invisible. Grid lines disappeared into the card background. Donut chart percentage labels in the default dark grey were unreadable. And perhaps most problematically, the bubble chart's default colorscale started at a dark color that blended into the dark card, making an entire cluster of data points invisible.

I tried addressing these issues one chart at a time initially, which produced inconsistent results and required me to repeat the same fixes multiple times. The solution that worked was building a shared `baseLayout()` function that applied consistent dark-theme overrides to every chart transparent paper and plot backgrounds, light-colored axis tick fonts, dark grid lines, and a transparent legend background. Any chart that needed additional overrides could pass them as an argument to be merged in. After building this function I only had to solve the dark-theme problem once, and every chart inherited the fix automatically.

What I learned was that reusable abstractions are worth the upfront investment even in small projects. The temptation when debugging visual issues is to fix them individually and move on. Building the `baseLayout()` function took an extra thirty minutes but saved me from resolving the same problem twelve separate times and produced a visually consistent result that would have been difficult to achieve any other way.

Section-8: AI-Assisted Development

Tools Used

I used Claude AI (Anthropic) as my primary AI assistant throughout the entire project lifecycle from initial scoping and proposal writing through data cleaning, JavaScript development, chart debugging, and documentation. I did not use GitHub Copilot or ChatGPT. All AI-assisted development in this project was conducted through Claude in the `claude.ai` interface.

Where AI Helped

The area where AI assistance was most consistently valuable was Python code generation for the data pipeline. Rather than looking up pandas' documentation for every transformation, I could describe the problem in plain English and receive working code immediately. For example, when I encountered the date parsing failure that was producing 1,646 null values, I described the problem directly: *"why is pd.to_datetime returning NaT for dates that look completely valid in my CSV format is 23-03-2023"*. The response immediately identified the `dayfirst=True` fix and flagged a follow-on issue I had not yet encountered those dates needed to be converted back to YYYY-MM-DD strings before JSON export for Plotly's time axis to recognize them correctly. That second piece of information saved me from a bug I would otherwise have hit an hour later.

AI was equally useful for JavaScript chart configuration. Plotly's API has extensive documentation, but it is dense and the dark-theme configuration requires knowing which specific layout properties to override. When I asked, *"how do I make Plotly charts look good on a dark background the default colors are all wrong"*, the response explained exactly which properties controlled which visual elements: `paper_bgcolor` and `plot_bgcolor` for backgrounds,

tickfont.color for axis labels, and legend.bgcolor for the legend. More importantly it suggested the baseLayout() pattern a shared function I could spread into every chart call which became the most reusable piece of code in the entire project.

For debugging, the most effective prompts were those that included the exact error message and the relevant code block together. Pasting the full traceback from a KeyError: 'category' error alongside the dataframe code immediately produced the correct diagnosis the column was called type in that specific CSV, not category.

tickfont.color for axis labels, and legend.bgcolor for the legend. More importantly it suggested the baseLayout() pattern a shared function I could spread into every chart call which became the most reusable piece of code in the entire project.

Where AI Struggled

AI assistance was least reliable when I asked about specific dataset characteristics. When I asked whether the Kaggle datasets I planned to use were clean and ready to use, the responses were accurate in general terms but could not account for the specific quirks of those particular files the type column instead of category, the GBP-formatted prices with pound symbols embedded in strings, or the fact that clean_apparel_by_brand.json was missing entirely. These issues only became visible by running the code against the real data. AI could help me fix each problem once I discovered it, but it could not anticipate problems it had never seen.

The other area where AI was not reliably useful was GitHub deployment troubleshooting. When I encountered the large file rejection error caused by the archive CSV files, the first suggested fix using git filter-branch produced a long command that partially worked but did not fully resolve the issue. The eventual solution was a combination of creating a .gitignore file and force pushing, which worked but required several iterations before the repository was clean. Git error messages are highly context-specific, and the correct fix depends on the exact state of the local repository, which AI cannot inspect directly.

Prompting Strategies

The most important thing I learned about prompting was that specificity produces dramatically better results than vague questions. Early in the project I would ask broad questions like *"how do I make my charts faster"* and receive generic suggestions. Later I started asking narrow, specific questions with context included: *"my clean_netaporter.json is 43,500 rows and the browser stalls loading it what is the most effective way to reduce file size while keeping the data the charts need"*. The second type of prompt produced actionable, targeted answers.

A second strategy that consistently worked well was providing the actual error message verbatim rather than describing it in my own words. Copying the full Python traceback or JavaScript console error into the prompt without paraphrasing gave the AI enough information to identify the root cause accurately rather than suggesting generic debugging steps.

The broader lesson I took from using AI throughout this project is that it functions best as a knowledgeable collaborator rather than an autonomous problem-solver. Every piece of code it generated required me to read it, understand it, test it against my actual data, and often modify it to fit the specific shape of my files and the specific requirements of my charts. The projects where AI assistance produces the best outcomes are the ones where the human understands the

domain well enough to evaluate whether the AI's suggestion is correct which means developing that understanding is still the most important investment

Section-9: User Testing & Feedback

I conducted informal user testing with five people three classmates from the FA 550 course, one friend with a business background but no data analysis experience, and one roommate with no technical background at all. The mix was intentional. I wanted to know whether the tool worked for people who already understand data visualization, but I also wanted to know whether someone with no technical context could open it and extract value without any guidance from me.

Each tester was given the live GitHub Pages link and three tasks to complete without my help:

- Find which product category generates the most revenue on the Sales Overview page
- Identify the luxury brand with the highest average price on the Brand Intelligence page
- Find which month has the highest search interest for any brand on the Brand Pulse page

I observed each session without intervening and noted where people hesitated, what they clicked first, and what confused them.

What worked well across all five testers was the navigation every person found and clicked the page tabs in the header immediately without any prompting. The KPI cards were also universally understood. Every tester looked at the KPI strip first before scrolling to the charts, which confirmed that putting headline numbers at the top of each page was the right layout decision. The year filter on Sales Overview was used unprompted by four out of five testers, which told me the filter placement and label were clear enough without instruction.

What did not work well fell into two consistent patterns. First, three testers were confused by the Brand Pulse heatmap on their first encounter. They understood it was showing something about months and brands but were unsure what the color intensity meant without reading the colorbar label. Second, two testers on the Brand Intelligence page initially tried to click on the treemap rectangles expecting something to happen a drill-down or a filter and were briefly confused when nothing did.

Changes I made based on feedback were targeted rather than comprehensive given the time available. For the heatmap confusion I added a subtitle beneath the chart title that reads "Darker gold = higher search interest" directly in the chart header, which resolved the issue in a follow-up check with two of the original testers. For the treemap click expectation I added a hover tooltip that appears immediately on mouse-over with the full brand name, product count, and average price making the chart feel more interactive even without click-through functionality. Both changes were small in code terms but meaningfully improved the experience for new users encountering the tool for the first time.

The most useful piece of feedback came from my non-technical roommate who said the tool felt "like something a real company would use" which was exactly the design intent. The one thing she could not interpret without help was the Google Trends relative scoring, which confirmed my documentation note about disclosing that limitation more prominently inside the tool itself.

Section-10: Reflection & Future Work

Personal Learning

Building Luxury Pulse from scratch taught me more than any other single project I have completed in this program, specifically because nothing was handed to me. There was no template, no starter code, and no instructor-provided dataset. Every decision from which data sources to use, to how to structure the JavaScript, to what colour the axis tick labels should be on a dark background was mine to make and mine to fix when it went wrong.

The most significant technical skill I developed was building a complete data pipeline independently. Before this project I had used pandas for assignments with clean, pre-formatted data. Here I had to diagnose silent failures, handle currency conversion, normalize inconsistent schemas across four different CSV formats, and export data in a structure optimized for browser performance. That end-to-end experience from raw CSV to deployed web application is something I can now speak about with genuine specificity in a job interview rather than in abstract terms.

I also developed a much more practical understanding of the relationship between data preparation and front-end performance. The decision of what to aggregate in Python versus what to compute in JavaScript is not just a technical preference it has a direct and measurable impact on how the tool feels to use. That lesson does not appear in any textbook I have read.

Project Evolution

The project changed substantially from my original proposal. I started with a plan to build a Tableau dashboard tracking financial performance across luxury conglomerates using data manually extracted from corporate annual reports. I ended up building a coded JavaScript tool powered by Kaggle datasets and a Google Trends API. The track changed, the tools changed, the data sources changed, and the scope was restructured twice.

If I were starting over, I would make the track switch decision in week one rather than week two. The time I spent trying to make the original plan work before accepting it was not feasible was time I could have spent building. I would also design the data pipeline with browser performance in mind from the beginning rather than retrofitting pre-aggregation after discovering the loading problem. The architecture of the JSON files what level of granularity to export, which summaries to pre-compute should be decided alongside the chart plan, not after the charts are already built.

Future Enhancements

The most meaningful addition I would make to Luxury Pulse is a geographic performance layer a filled world map showing which regions drive the most revenue and which markets have the fastest growing brand search interest. This was in my original proposal and was cut for time. The data exists in the Google Trends API by country parameter and could be pulled with a minor modification to the existing pipeline.

I would also add cross-page filtering so that selecting a brand in the Brand Intelligence treemap automatically filters the Brand Pulse page to show that brand's search interest data. This kind of linked interactivity is what separates a tool from a collection of charts, and it is technically straightforward using the existing JavaScript architecture.

Additional data sources I would incorporate include real-time stock price data via yfinance for publicly traded luxury conglomerates, which would add a market sentiment dimension that currently does not exist in the tool, and social media engagement metrics for the tracked brands, which would let me compare search interest from Google Trends against social media engagement on Instagram or TikTok.

Broader Applications

The architecture of Luxury Pulse is not specific to fashion retail. The same structure, a Python cleaning pipeline exporting pre-aggregated JSON files consumed by a browser-based Plotly dashboard with page-level filtering could be applied to any industry with multiple data sources that benefit from being analyzed together. A healthcare version could track patient admission trends, medication costs, and regional access patterns. A sports analytics version could combine game performance data, ticket sales, and social media sentiment for a professional team. A financial services version could track portfolio performance, market indices, and news sentiment simultaneously.

The broader lesson I take from this project is that the barrier to building a genuinely useful data tool is lower than most people assume. No enterprise software licence, no cloud infrastructure, and no framework expertise are required. A Python script, a folder of JSON files, and a well-structured JavaScript application deployed on GitHub Pages can produce something that looks professional, loads quickly, and surfaces real insights and that anyone in the world can open with a single link.

I pledge on my honor that I have not given or received any unauthorized assistance on this assignment/examination. I further pledge that I have not copied any material from a book, article, the Internet or any other source except where I have expressly cited the source. I have documented all AI tool usage including prompts and tool selection rationale.

Signature: _____Roshni Patel_____

Date: _____15/09/26_____